

**JAVA**

# Lambda Expressions & Functional Interfaces

Consumer • Predicate • Function • Supplier • Method References

Apostila completa com explicações detalhadas e exemplos práticos para consolidar programação funcional em Java.

## CONTEÚDO DA APOSTILA

- 01** Introdução às Lambda Expressions
- 02** Sintaxe da Lambda Expression
- 03** Functional Interfaces
- 04** Consumer — void accept(T)
- 05** Predicate — boolean test(T)
- 06** Function e Supplier
- 07** Method References
- 08** Chaining e Comparator

# Sumário

- 01 Introdução às Lambda Expressions**  
O que são e a relação com Anonymous Classes

---

- 02 Sintaxe da Lambda Expression**  
Parâmetros, arrow token e variações de corpo

---

- 03 Functional Interfaces**  
SAM, @FunctionalInterface e inferência de contexto

---

- 04 Consumer — void accept(T)**  
Execução sem retorno e encadeamento com andThen

---

- 05 Predicate — boolean test(T)**  
Filtros booleanos com and, or e negate

---

- 06 Function e Supplier**  
apply, compose, andThen e inicialização preguiçosa

---

- 07 Method References**  
Os quatro tipos de referência com ::

---

- 08 Chaining e Comparator**  
Combinando comportamentos de forma fluente

# 01 Introdução às Lambda Expressions

*O que são, execução diferida e a relação com Anonymous Classes*

Introduzidas no JDK 8, as lambda expressions estão entre os recursos que mais mudaram o estilo de escrever Java moderno. A forma mais simples de pensar em uma lambda é como um atalho compacto para uma Anonymous Class que implementa uma interface funcional. Enquanto a Anonymous Class exige toda a cerimônia de declarar uma classe — `new`, `@Override`, chaves — a lambda reduz tudo isso a parâmetros, uma seta e um corpo.

Uma propriedade essencial de qualquer lambda é a execução diferida: o código escrito dentro dela não roda no momento em que é declarado. Ele fica apenas guardado, pronto para ser executado mais tarde, quando o método funcional da interface for finalmente chamado — seja `accept()`, `test()`, `apply()` ou `get()`.

```
// Anonymous Class (antes do JDK 8):
Comparator<String> porNome = new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return a.compareToIgnoreCase(b);
    }
};

// lambda equivalente (JDK 8+):
Comparator<String> porNome2 = (a, b) -> a.compareToIgnoreCase(b);

// method reference, ainda mais direto:
Comparator<String> porNome3 = String::compareToIgnoreCase;
```

*Exemplo — a mesma comparação escrita de três formas diferentes*

## DICA

Lembre-se: uma lambda não executa nada no ponto em que é declarada — é execução diferida. O código só roda quando o método funcional da interface é efetivamente chamado. Isso é o que permite passar comportamento como se fosse um dado, guardando-o para usar mais tarde.

# 02 Sintaxe da Lambda Expression

*Parâmetros, o arrow token e as variações de corpo*

Toda lambda tem três partes: os parâmetros entre parênteses, o arrow token `->` — sempre uma seta, nunca `=>` — e o corpo, que pode ser uma expressão simples ou um bloco entre chaves. O compilador infere o tipo dos parâmetros a partir da interface funcional alvo, então raramente é preciso declará-los explicitamente.

```
// variações de parâmetros:
() -> expressao // zero parâmetros → () obrigatório
x -> expressao // um parâmetro sem tipo → () opcional
(String x) -> expressao // tipo explícito → () obrigatório
(x, y) -> expressao // dois ou mais parâmetros → () obrigatório

// corpo – expressão simples (sem 'return', sem chaves):
(a, b) -> a.compareToIgnoreCase(b) // retorno implícito

// corpo – bloco com chaves (return obrigatório se houver retorno):
(a, b) -> {
    System.out.println("comparando...");
    return a.compareToIgnoreCase(b);
}
```

Exemplo — as principais variações de sintaxe de uma lambda

| Tipo de corpo               | Regra de retorno                       |
|-----------------------------|----------------------------------------|
| Expressão simples com valor | Retorno implícito — sem usar return    |
| Expressão simples void      | Nenhum retorno — o método é void       |
| Bloco com retorno           | return é obrigatório dentro das chaves |
| Bloco sem retorno           | Método void — return não é necessário  |

#### ATENÇÃO

Se você declarar o tipo de um parâmetro explicitamente, todos os demais parâmetros da mesma lambda também precisam ter tipo declarado. Misturar tipo inferido com tipo explícito no mesmo grupo de parênteses é erro de compilação.

## 03 Functional Interfaces

*SAM, @FunctionalInterface e como o Java infere o método alvo*

Uma functional interface é qualquer interface com exatamente um método abstrato — chamado de SAM (Single Abstract Method), ou método funcional. É esse método que toda lambda escrita para aquela interface está implementando. O Java usa o tipo declarado da variável ou do parâmetro para decidir qual interface (e, portanto, qual método) a lambda representa — um processo chamado inferência de contexto.

A anotação `@FunctionalInterface` é opcional, mas muito recomendada: ela instrui o compilador a verificar que a interface tem exatamente um método abstrato, emitindo erro caso um segundo método abstrato seja adicionado sem querer. Métodos `default` e `static` não contam nessa verificação.

```
@FunctionalInterface
public interface Transformador {
    String transformar(String entrada); // único método abstrato – o SAM

    // método default não conta como abstrato:
    default String transformarComLog(String s) {
        System.out.println("Transformando: " + s);
        return transformar(s);
    }
}

// como o Java infere o método alvo:
// 1. maiuscula é do tipo Transformador
// 2. Transformador tem 1 SAM: transformar(String) -> String
// 3. logo, a lambda implementa transformar
Transformador maiuscula = s -> s.toUpperCase();
Transformador semEspacos = String::trim; // method reference
```

*Exemplo — uma funcional interface com um método default e duas lambdas*

O pacote `java.util.function` já traz mais de 40 interfaces funcionais prontas, organizadas em quatro categorias principais: `Consumer`, `Predicate`, `Function` e `Supplier`. Elas cobrem a grande maioria dos casos de uso do dia a dia — raramente será necessário criar uma interface funcional própria.

## 04 Consumer — void accept(T)

*Executar uma ação sem retornar resultado*

A interface `Consumer<T>` representa uma operação que recebe um argumento do tipo `T` e não devolve nenhum resultado. É o tipo ideal para efeitos colaterais: imprimir, salvar em um arquivo, enviar uma notificação, registrar um log. O método principal é `void accept(T t)`.

O método de conveniência `andThen(Consumer c2)` encadeia dois `Consumers`: primeiro executa o atual, depois executa `c2`, ambos recebendo o mesmo argumento. É diferente do `andThen` de `Function`, que passa o resultado de um para o próximo — aqui, cada `Consumer` atua de forma independente sobre o mesmo valor de entrada.

```
// Consumers simples:
Consumer<String> imprimir = s -> System.out.println(s);
Consumer<String> gritar = s -> System.out.println(s.toUpperCase());

imprimir.accept("pedido recebido"); // pedido recebido
gritar.accept("pedido recebido"); // PEDIDO RECEBIDO

// andThen – encadeia Consumers sobre o MESMO argumento:
Consumer<String> imprimirEGritar = imprimir.andThen(gritar);
imprimirEGritar.accept("novo cliente");
// novo cliente
// NOVO CLIENTE

// BiConsumer<T,U> – recebe dois argumentos:
BiConsumer<String, Double> registrarVenda =
(prodoto, valor) -> System.out.println(prodoto + " – R$" + valor);
registrarVenda.accept("Teclado", 189.90);
```

*Exemplo — Consumer simples, encadeamento com andThen e BiConsumer*

## 05 Predicate — boolean test(T)

*Testar condições, compor filtros e usar com removeIf*

A interface `Predicate<T>` representa uma condição que recebe um argumento e devolve `true` ou `false`. É perfeita para filtros, validações e testes, sendo amplamente usada com `Collection.removeIf()` e com `stream.filter()`.

Os métodos de composição são o grande diferencial do `Predicate`: `and(p2)` retorna verdadeiro só quando ambos os predicados são verdadeiros; `or(p2)` retorna verdadeiro quando pelo menos um deles é; e `negate()` inverte o resultado. Todos usam short-circuit evaluation, o que os torna eficientes mesmo em cadeias longas.

```
// predicados simples:
Predicate<String> temEmail = s -> s.contains("@");
Predicate<String> ehLongo = s -> s.length() > 20;
Predicate<Integer> ehMaiorDeIdade = idade -> idade >= 18;

// composição de predicados:
Predicate<String> emailLongo = temEmail.and(ehLongo);
Predicate<String> emailOuLongo = temEmail.or(ehLongo);
Predicate<String> semEmail = temEmail.negate();

// removeIf – remove elementos que satisfazem o predicado:
List<String> tags = new ArrayList<>(List.of("a", "bb", "ccc", "dddd"));
tags.removeIf(s -> s.length() < 3); // remove "a" e "bb"
System.out.println(tags); // [ccc, dddd]
```

*Exemplo — Predicate simples, composição e uso com removeIf*

## 06 Function e Supplier

*Transformar valores e produzir instâncias sob demanda*

### Function — R apply(T t)

A interface `Function<T, R>` representa uma transformação: recebe um valor do tipo `T` e devolve um resultado do tipo `R`, que pode ser diferente. `UnaryOperator` é o caso especial em que entrada e saída têm o mesmo tipo; `BinaryOperator` recebe dois valores do mesmo tipo e devolve um valor também daquele tipo.

O método `andThen(f2)` cria um pipeline: executa a `Function` atual e passa o resultado para `f2`. Já `compose(f1)` faz o inverso — executa `f1` primeiro, e só depois aplica a `Function` atual sobre o resultado.

```
Function<String, Integer> tamanho = String::length;
Function<String, String[]> dividir = s -> s.split(",");

tamanho.apply("pedido"); // 6
dividir.apply("a,b,c"); // ["a", "b", "c"]

// UnaryOperator – mesmo tipo de entrada e saída:
UnaryOperator<String> semEspacos = String::trim;
UnaryOperator<String> maiusculas = String::toUpperCase;

// andThen – pipeline: primeiro semEspacos, depois maiusculas:
Function<String, String> normalizar = semEspacos.andThen(maiusculas);
normalizar.apply(" ana silva "); // "ANA SILVA"
```

*Exemplo — Function, UnaryOperator e um pipeline com andThen*

### Supplier — T get()

A interface `Supplier<T>` é a mais simples de todas: não recebe nenhum argumento e devolve uma instância de `T`. Funciona como um método de fábrica — produz objetos sob demanda — e é ideal para inicialização preguiçosa (lazy initialization), quando a criação de algo custoso deve acontecer só quando `get()` é efetivamente chamado.

```
Supplier<String> saudacao = () -> "Bem-vindo de volta!";
saudacao.get(); // só executa agora

// factory method com method reference:
Supplier<ArrayList<String>> fabrica = ArrayList::new;
ArrayList<String> lista1 = fabrica.get(); // nova lista
ArrayList<String> lista2 = fabrica.get(); // outra nova lista

// lazy initialization – a criação só ocorre quando necessária:
Supplier<ConexaoBanco> conexao = () -> new ConexaoBanco("jdbc:...");
ConexaoBanco c = conexao.get(); // criada apenas aqui
```

Exemplo — Supplier como factory method e em inicialização preguiçosa

## 07 Method References

A sintaxe :: e os quatro tipos, do estático ao construtor

Um method reference é uma forma ainda mais enxuta de escrever uma lambda que apenas chama um método já existente. Usa a sintaxe :: entre o tipo (ou a instância) e o nome do método. O compilador infere parâmetros e retorno a partir da interface funcional alvo — o resultado é exatamente equivalente a escrever a lambda por extenso.

| Tipo          | Sintaxe                   | Exemplo             | Lambda equivalente         |
|---------------|---------------------------|---------------------|----------------------------|
| Estático      | Classe::metodoEstatico    | Integer::parseInt   | s -> Integer.parseInt(s)   |
| Vinculado     | instancia::metodo         | System.out::println | s -> System.out.println(s) |
| Não vinculado | Classe::metodoDeInstancia | String::toUpperCase | s -> s.toUpperCase()       |
| Construtor    | Classe::new               | ArrayList::new      | () -> new ArrayList<>()    |

### O receiver não vinculado — o ponto mais confuso

O tipo não vinculado parece uma referência estática (Classe::metodo), mas na verdade é um método de instância em que o primeiro parâmetro da interface funcional vira a instância que chama o método. Por exemplo, String::toUpperCase equivale a s -> s.toUpperCase() — a própria String s é a instância, não uma classe estática sendo chamada.



```
// não vinculado: a instância vem do primeiro argumento:
UnaryOperator<String> paraMaiuscula = String::toUpperCase;
paraMaiuscula.apply("relatorio"); // "relatorio".toUpperCase() -> "RELATORIO"

// String::concat — 1º parâmetro é a instância, 2º é o argumento:
BinaryOperator<String> concatenar = String::concat;
concatenar.apply("Olá, ", "mundo!"); // "Olá, mundo!"

// comparando os dois casos:
// ESTÁTICO: Integer::sum -> (a,b) -> Integer.sum(a,b)
// NÃO VINCULADO: String::concat -> (s1,s2) -> s1.concat(s2)
```

Exemplo — a distinção entre *method reference* estático e não vinculado

## 08 Chaining e Comparator

*Combinando comportamentos de forma fluente e declarativa*

Os métodos de conveniência das interfaces funcionais são o que torna as lambdas realmente poderosas: eles permitem combinar comportamentos de forma fluente, sem escrever lógica condicional explícita. O código resultante descreve o que deve acontecer, não como — muito mais próximo de uma especificação do que de uma implementação passo a passo.

```
// pipeline de Function:
UnaryOperator<String> semEspacos = String::trim;
UnaryOperator<String> maiusculas = String::toUpperCase;
Function<String, String[]> dividir = s -> s.split(",");

var pipeline = semEspacos.andThen(maiusculas).andThen(dividir);
pipeline.apply(" ana, bruno "); // ["ANA", " BRUNO"]

// composição de Predicate em um filtro de stream:
Predicate<String> naoVazio = s -> !s.isBlank();
Predicate<String> comeceComA = s -> s.startsWith("A");
Predicate<String> curto = s -> s.length() <= 6;

List.of("Ana", "", "Alberto", "Al").stream()
    .filter(naoVazio.and(comeceComA).and(curto))
    .forEach(System.out::println); // Ana, Al
```

Exemplo — *pipeline de Function* e *composição de Predicate*

### Comparator com lambdas

Desde o Java 8, `Comparator` ganhou métodos estáticos e de instância bastante úteis. `Comparator.comparing(extrator)` cria um comparador baseado em um campo; `thenComparing()` adiciona um critério de desempate; e `reversed()` inverte a ordem — todos retornam um novo `Comparator`, permitindo encadeamento fluente.

```
// ordenar por sobrenome e, em empate, por nome:
Comparator<Cliente> porSobrenomeDepoisNome =
    Comparator.comparing(Cliente::getSobrenome)
        .thenComparing(Cliente::getNome);

// ordem decrescente:
Comparator<Cliente> porSobrenomeDesc =
    Comparator.comparing(Cliente::getSobrenome).reversed();

clientes.sort(porSobrenomeDepoisNome);
```

*Exemplo — encadeando critérios de ordenação com Comparator*

## Resumo Final

|                             |                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------|
| <b>Lambda</b>               | Atalho conciso para uma Anonymous Class de uma interface funcional; execução diferida. |
| <b>Functional Interface</b> | Interface com exatamente um método abstrato (SAM).                                     |
| <b>Consumer / Predicate</b> | Executa uma ação sem retorno / testa uma condição booleana.                            |
| <b>Function / Supplier</b>  | Transforma um valor em outro / produz um valor sob demanda.                            |
| <b>Method Reference</b>     | Sintaxe :: para apontar direto a um método já existente.                               |